

אינטגרציית SmartConsole מול SolarWinds מוסבר

Check Point ≠ SolarWinds · סקריפטים 4

מסמך זה עובר על כל אחד מארבעת הסקריפטים לפי סדר ההופעה בקוד, ומסביר מה כל בלוק קוד עושה ולמה הוא בנוי בצורה הזו — מהתחברות ל-Management API של Check Point, דרך הפונקציה המשותפת ליצירת Node ב-SolarWinds, ועד שני מסלולי הצריכה: CSV מול אורקסטרטור.

PowerShell קבצי 4

10 סעיפים

branch: claude/smartconsole-api-integration

מה האוטומציה הזו עושה

המטרה היא לקחת את כתובות ה-IP של חברי הפיירוול ששייכים לאובייקט ClusterXL (מסוג `simple-cluster`) בשרת הניהול של Check Point, ולרשום כל חבר ככזה כ-Node מנוטר ב-SolarWinds. כדי לעשות את זה בצורה אמינה צריך לדבר עם שתי מערכות שאין ביניהן קשר, לכל אחת שיטת אימות וסגנון API משלה — ולכן העבודה מפוצלת לארבעה סקריפטים ולא לאחד גדול:

סקריפט	תפקיד
<code>CheckPoint.GetClusterFirewallAddresses.ps1</code>	מתחבר ל-Management API של Check Point (ה-API שמאחורי SmartConsole) ומחזיר את כתובות ה-IP של חברי הקלאסטר.
<code>CRUD.AddNodeWithPollers.ps1</code>	פונקציית עזר משותפת: מוסיפה Node אחד ב-SolarWinds ורושמת לו את הפולרים המובנים בברירת המחדל.
<code>CRUD.AddNodesFromCsv.ps1</code>	צרכן מס' 1 — קורא קובץ CSV (זוגות שם/IP) ומוסיף כל שורה כ-Node ב-SolarWinds.
<code>Orchestrator.AddClusterFirewallsToSolarWinds.ps1</code>	צרכן מס' 2 — קורא ישירות לפונקציית הגילוי ומזרים את התוצאות ישיר ל-SolarWinds, בלי קובץ ביניים.

למה ארבעה קבצים ולא אחד

- **לוחות זמנים וביקורת עצמאיים.** הגילוי יכול לרוץ בקצב משלו, ואת הרשימה שמתקבלת אפשר לבדוק לפני שמוסיפים משהו לניטור.
 - **בדיקה עצמאית.** כל צד אפשר להריץ (או לדמות) בלי לצטרך את פרטי ההתחברות של הצד השני.
 - **שימוש חוזר.** ל-`Add-SwisNodeWithDefaultPollers` לא אכפת מהיכן הגיעה כתובת ה-IP — זו אותה פונקציה בדיוק שמשמשת גם את `CRUD.AddNodesFromCsv.ps1` וגם את האורקסטרטור.
- שני סקריפטי הצריכה מייצגים את שני הקצוות של הפשרה הזו: מבוסס-CSV לשער ביקורת ידני, מבוסס-אורקסטרטור לריצה אוטומטית אחת (למשל משימה מתוזמנת).

CheckPoint.GetClusterFirewallAddresses.ps1

התחברות ל-Management API של Check Point וגילוי חברי הקלאסטר

```
14 function Connect-SmartConsole {
15     param(
16         [Parameter(Mandatory = $true)] [string]$Server,
17         [int]$Port = 443,
18         [Parameter(Mandatory = $true)] [string]$Username,
19         [Parameter(Mandatory = $true)] [System.Security.SecureString]$Password,
20         [switch]$TrustAllCertificates
21     )
22
23     $plainPassword = [System.Runtime.InteropServices.Marshal]::PtrToStringAuto(
24         [System.Runtime.InteropServices.Marshal]::SecureStringToBSTR($Password)
25     )
26
27     $body = @{
28         user      = $Username
29         password = $plainPassword
30     } | ConvertTo-Json
31
32     $uri = "https://{Server}:{Port}/web_api/v1.9/login"
33
34     $params = @{
35         Uri      = $uri
36         Method   = "Post"
37         Body     = $body
38         ContentType = "application/json"
39     }
40     if ($TrustAllCertificates) {
41         $params["SkipCertificateCheck"] = $true
42     }
43
44     $response = Invoke-RestMethod @params
45
46     [pscustomobject]@{
47         Server      = $Server
48         Port        = $Port
49         Sid         = $response.sid
50         TrustAllCertificates = [bool]$TrustAllCertificates
51     }
52 }
```

מתחברת עם `POST /web_api/v1.9/login` (שורה 32), ושולחת את שם המשתמש ועותק מפוענח-בזיכרון של הסיסמה שהגיעה כ- `SecureString` (שורות 23-25). השרת מחזיר מזהה session (`sid`), שמוחזר בתוך אובייקט session קטן (`Server, Port, Sid, TrustAllCertificates`, שורות 46-51) שכל קריאה מאוחרת יותר צריכה.

תעודות SSL: כברירת מחדל הפונקציה לא עוקפת אימות TLS. שרתי ניהול של Check Point בדרך כלל מציגים תעודה חתומה עצמית, ולכן יש switch בשם `-TrustAllCertificates` (שורות 40-42, 20) — אבל זו בחירה מפורשת (opt-in) ולא ברירת מחדל.

session-ה — Disconnect-SmartConsole

2

```
54 function Disconnect-SmartConsole {
55     param(
56         [Parameter(Mandatory = $true)] [pscustomobject]$Session
57     )
58
59     $uri = "https://$(($Session.Server):$(($Session.Port)/web_api/v1.9/logout"
60     $params = @{
61         Uri          = $uri
62         Method       = "Post"
63         Headers      = @{ "X-chkp-sid" = $Session.Sid }
64         Body         = "{}"
65         ContentType  = "application/json"
66     }
67     if ($Session.TrustAllCertificates) {
68         $params["SkipCertificateCheck"] = $true
69     }
70
71     Invoke-RestMethod @params | Out-Null
72 }
```

קוראת ל- `logout` (שורה 59) כדי לבטל את מזהה ה-session. חשוב לקרוא לזה תמיד בתוך בלוק `finally` בקוד הקורא, כדי שגילוי שנכשל לא ישאיר session פתוח דולף.

```

74 function Invoke-SmartConsoleApi {
75     param(
76         [Parameter(Mandatory = $true)] [pscustomobject]$Session,
77         [Parameter(Mandatory = $true)] [string]$Command,
78         [hashtable]$Body = @{}
79     )
80
81     $uri = "https://$($Session.Server):$($Session.Port)/web_api/v1.9/$Command"
82     $params = @{
83         Uri          = $uri
84         Method       = "Post"
85         Headers      = @{ "X-chkp-sid" = $Session.Sid }
86         Body         = ($Body | ConvertTo-Json -Depth 10)
87         ContentType  = "application/json"
88     }
89     if ($Session.TrustAllCertificates) {
90         $params["SkipCertificateCheck"] = $true
91     }
92
93     Invoke-RestMethod @params
94 }

```

עטיפה דקה סביב `Invoke-RestMethod` שדרכה עוברת כל קריאת API אחרת: היא בונה את ה-URL משם הפקודה (שורה 81), מקודדת את הגוף כ-JSON (שורה 86), ומצרפת את מזהה ה-session כותרת `X-chkp-sid` (שורה 85) שה-API דורש.

```

96 function Get-SmartConsoleClusterFirewallAddresses {
97     param(
98         [Parameter(Mandatory = $true)] [pscustomobject]$Session
99     )
100
101     $limit = 50
102     $offset = 0
103     $results = @()
104
105     do {
106         $page = Invoke-SmartConsoleApi -Session $Session -Command "show-simple-clusters"
107         -Body @{
108             limit = $limit
109             offset = $offset
110         }
111
112         foreach ($clusterSummary in $page.objects) {
113             $cluster = Invoke-SmartConsoleApi -Session $Session -Command "show-simple-
114             cluster" -Body @{
115                 uid = $clusterSummary.uid
116                 "details-level" = "full"
117             }
118
119             foreach ($member in $cluster.'cluster-members') {
120                 $results += [pscustomobject]@{
121                     ClusterName = $cluster.name
122                     ClusterVip = $cluster.'ipv4-address'
123                     MemberName = $member.name
124                     MemberIp = $member.'ip-address'
125                 }
126             }
127
128             $offset += $limit
129         } while ($offset -lt $page.total)
130
131     } $results
132 }

```

אובייקטי ClusterXL של Check Point מיוצגים על-ידי סוג האובייקט `simple-cluster`. הפונקציה קוראת ל-`show-simple-clusters` (שורה 106) כדי לעבור על כל אובייקטי הקלאסטר בדפדוף — 50 בכל פעם, לפי מוסכמת הדפדוף `total / offset / limit` של ה-API (שורות 101-102, 127-128). עבור כל קלאסטר, היא קוראת ל-`show-simple-cluster` עם `details-level=full` (שורות 112-115) כדי לקבל את רשימת החברים המלאה, ומשטחת את התוצאה: שורת פלט אחת לכל חבר קלאסטר (שורות 117-124), שנושאת את שם הקלאסטר ואת ה-VIP שלו לצד השם וכתובת ה-IP של אותו חבר עצמו.

MemberIp / MemberName הם בדיוק שמות העמודות ששני סקריפטי הצריכה ב-SolarWinds מצפים להם —
השיתוף הזה בשמות הוא מה שמאפשר גם למסירה דרך CSV וגם למסירה הישירה דרך האורקסטרטור לעבוד בלי שלב
תרגום.

CRUD.AddNodeWithPollers.ps1

פונקציית העזר המשותפת ליצירת Node ב-SolarWinds

הגדרת הפרמטרים ותכונות ה-Node

5

```

6 function Add-SwisNodeWithDefaultPollers {
7     param(
8         [Parameter(Mandatory = $true)] $Swis,
9         [Parameter(Mandatory = $true)] [string]$IPAddress,
10        [string]$Name = "",
11        [string]$Community = "public",
12        [string]$RWCommunity = ""
13    )
14
15    $newNodeProps = @{
16        IPAddress = $IPAddress;
17        EngineID = 1;
18        Caption = $Name;
19
20        # SNMP v2 specific
21        ObjectSubType = "SNMP";
22        SNMPVersion = 2;
23        Community = $Community;
24        RWCommunity = $RWCommunity;
25
26        DNS = "";
27        SysName = "";
28    }

```

פונקציה אחת, `Add-SwisNodeWithDefaultPollers`, שהוצאה מתוך דוגמת ה-`CRUD.AddNode.ps1` המקורית כדי ששני סקריפטי הצריכה יוכלו לקרוא לה במקום כל אחד לשכפל בעצמו את אותן קריאות `SWIS.Community` (שורה 11, 23) הוא מחרוזת ה-`community` לקריאה (SNMP v2) ש-SolarWinds משתמש בה כדי לתשאל את המכשיר; ברירת המחדל היא `"public"`, אבל זה פרמטר כדי שפיירוולים אמיתיים (שלעולם לא צריך להשאיר על ה-`community` המחדל) יוכלו להעביר משלהם. `RWCommunity` (שורה 12, 24) היא ה-`community` לכתובה, אופציונלית, ונשארת ריקה אלא אם SolarWinds צריך גם להגדיר את המכשיר, לא רק לנטר אותו.

```
30 $newNodeUri = New-SwisObject $Swis -EntityType "Orion.Nodes" -Properties $newNodeProps
31 $nodeProps = Get-SwisObject $Swis -Uri $newNodeUri
32
33 $poller = @{
34     NetObject="N:"+$nodeProps["NodeID"];
35     NetObjectType="N";
36     NetObjectID=$nodeProps["NodeID"];
37 }
38
39 foreach ($pollerType in @(
40     "N.Status.ICMP.Native",
41     "N.ResponseTime.ICMP.Native",
42     "N.Details.SNMP.Generic",
43     "N.Uptime.SNMP.Generic"
44 )) {
45     $poller["PollerType"] = $pollerType
46     New-SwisObject $Swis -EntityType "Orion.Pollers" -Properties $poller | Out-Null
47 }
48
49 $nodeProps
50 }
```

נדרשות שתי קריאות SWIS לכל Node: `New-SwisObject` (שורה 30) ואז `Get-SwisObject` (שורה 31), כי קריאת היצירה מחזירה רק URI — ה-NodeID שמשמש לרישום הפולרים (שורות 39-47) צריך להישלף בחזרה אחר-כך.

CRUD.AddNodesFromCsv.ps1

הצרכן ה-מנותק: מוסיף Node-ים מקובץ CSV שנבדק מראש

פרמטרים, dot-source וחיבור ל-SWIS

7

```
16 param(
17     [string]$CsvPath = ".\ClusterFirewalls.csv",
18     [string]$NameColumn = "MemberName",
19     [string]$IpColumn = "MemberIp",
20     [string]$Community = "public"
21 )
22
23 # Dot-source the shared node-creation helper (Add-SwisNodeWithDefaultPollers).
24 . (Join-Path $PSScriptRoot "CRUD.AddNodeWithPollers.ps1")
25
26 # Connect to SWIS
27 $hostname = "localhost"
28 $username = "admin"
29 $password = New-Object System.Security.SecureString
30 $cred = New-Object -typename System.Management.Automation.PSCredential -argumentlist
31 $username, $password
32
33 $swis = Connect-Swis -host $hostname -cred $cred
34
35 $rows = Import-Csv -Path $CsvPath
```

שמות העמודות (`-NameColumn` / `-IpColumn` , שורות 18-19) הם פרמטרים ולא קבועים בקוד, כך שאותו סקריפט עובד מול CSV מכל מקור, לא רק מזה של Check Point. שורה 24 עושה `dot-source` לקובץ הפונקציה המשותפת, כדי שהפונקציה `Add-SwisNodeWithDefaultPollers` תהיה זמינה כאן.

```

35 foreach ($row in $rows) {
36     $name = $row.$NameColumn
37     $ip   = $row.$IpColumn
38
39     if ([string]::IsNullOrEmpty($ip)) {
40         Write-Warning "Skipping row with no IP address (name: '$name')."
41         continue
42     }
43
44     Write-Host "Adding node '$name' ($ip)..."
45     Add-SwisNodeWithDefaultPollers -Swis $swis -Name $name -IPAddress $ip -Community
46     $Community | Out-Null
47 }

```

קורא קובץ CSV (כברירת מחדל `.ClusterFirewalls.csv`) — הקובץ שה- `Export-Csv` בדוגמת השימוש של סקריפט ה-SmartConsole מייצר) וקורא לפונקציה המשותפת פעם אחת עבור כל שורה (שורה 45), תוך דילוג על שורות בלי כתובת IP (שורות 39-42).

מכיוון שה-CSV יושב בין שתי המערכות כקובץ פשוט, הוא משמש גם כתיעוד ביקורת (audit trail): אפשר לפתוח אותו, למחוק שורה שלא רוצים לנטר, או להשוות אותו לקובץ של אתמול לפני שמריצים את הסקריפט הזה.

Orchestrator.AddClusterFirewallsToSolarWinds.ps1

הצרכן ל-ריצה אחת: מגלה ומוסיף בלי קובץ ביניים

Check Point והתחברות ל-Dot-sourcing

9

```

21 . (Join-Path $PSScriptRoot "CheckPoint.GetClusterFirewallAddresses.ps1")
22 . (Join-Path $PSScriptRoot "CRUD.AddNodeWithPollers.ps1")
23
24 # --- Check Point SmartConsole connection ---
25 $cpServer = "smartcenter.example.com"
26 $cpUsername = "admin"
27 $cpPassword = Read-Host -Prompt "Check Point password" -AsSecureString
28
29 # --- SolarWinds connection ---
30 $swHostname = "localhost"
31 $swUsername = "admin"
32 $swPassword = New-Object System.Security.SecureString
33 $swCred = New-Object -typename System.Management.Automation.PSCredential -argumentlist
34 $swUsername, $swPassword
35 $swCommunity = "public"
36
37 $cpSession = Connect-SmartConsole -Server $cpServer -Username $cpUsername -Password
38 $cpPassword
39
40 try {
41     $firewalls = Get-SmartConsoleClusterFirewallAddresses -Session $cpSession
42 }
43 finally {
44     Disconnect-SmartConsole -Session $cpSession
45 }

```

שורות 21-22 עושות `dot-source` לשני קבצי הפונקציות — הנקודה בתחילת השורה מריצה את הקובץ היעד ב-`scope` הנוכחי במקום ב-`scope` ילד, כך שהפונקציות שהם מגדירים (`Get-`, `Connect-SmartConsole`) הופכות לזמינות לקריאה כאן. אף אחד מהקבצים שעושים להם `source` לא מריץ את בלוק דוגמת השימוש שלו, כי הבלוק הזה מוער בתחתית כל קובץ. שורות 36-43 מתחברות, מריצות את הגילוי בתוך `try`, ומתנתקות תמיד בתוך `finally` — גם אם הגילוי נכשל.

```
45 if (-not $firewalls) {
46     Write-Warning "No cluster firewall addresses were discovered - nothing to add."
47     return
48 }
49
50 $swis = Connect-Swis -host $swHostname -cred $swCred
51
52 foreach ($fw in $firewalls) {
53     if ([string]::IsNullOrEmpty($fw.MemberIp)) {
54         Write-Warning "Skipping '$($fw.MemberName)' - no IP address returned."
55         continue
56     }
57
58     Write-Host "Adding node '$($fw.MemberName)' ($($fw.MemberIp)) from cluster '$($fw.ClusterName)'..."
59     Add-SwisNodeWithDefaultPollers -Swis $swis -Name $fw.MemberName -IPAddress
    $fw.MemberIp -Community $swCommunity | Out-Null
60 }
```

מתחברת ל-SWIS (שורה 50) ועוברת על כל חבר קלאסטר שהתגלה, וקוראת ל- **Add-** **SwisNodeWithDefaultPollers** (שורה 59) בדיוק כמו סקריפט ה-CSV.

הסקריפט הזה צריך גם את פרטי ההתחברות ל-Check Point וגם את אלה של SolarWinds זמינים באותה ריצה, והוא מוסיף Node-ים ברגע שהם מתגלים — אין שלב ביקורת. כדאי להשתמש במסלול ה-CSV כשהפער הזה חשוב (למשל ניטור בפרודקשן).

הזרימה מקצה לקצה, שני המסלולים

שלב	מסלול עם ביקורת (CSV)	מסלול אוטומטי (אורקסטרטור)
1	הרצת <code>Export-Csv</code> , <code>Checkpoint.GetClusterFirewallAddresses.ps1</code> לתוצאה	הרצת סקריפט האורקסטרטור
2	פתיחה/בדיקה של <code>ClusterFirewalls.csv</code>	(הגילוי קורה בזיכרון, אין שלב ביקורת)
3	הרצת <code>CRUD.AddNodesFromCsv.ps1 -CsvPath ClusterFirewalls.csv</code>	ה-Node ים נוספים מיד
4	Node ים ופולרים נוצרים ב-SolarWinds	Node ים ופולרים נוצרים ב-SolarWinds

דברים לשנות לפני הרצה אמיתית

- למלא שמות שרתים/פרטי התחברות אמיתיים ל-Check Point ול-SolarWinds — כל סקריפט מגיע עם ערכי placeholder ומצפה שתערכו אותם.
- להגדיר מחרוזת SNMP community אמיתית לכל סביבה — `"public"` היא ברירת מחדל, לא המלצה.
- להחליט במפורש על `-TrustAllCertificates` אם שרת הניהול של Check Point משתמש בתעודה חתומה עצמית.